

Programación declarativa

• Haskell •

Doble grado en Ingeniería Informática y Matemáticas

Universidad Complutense de Madrid

Óscar Martín — Curso 2026/27

8. Definición de funciones por casos

Patrones

- Véase el tema correspondiente.

if-then-else

```
mayor :: Ord a => a -> a -> a
mayor x y = if x > y then x else y
```

- El **if-then-else** es una expresión, no una instrucción.
 - Siempre tiene valor.
 - No existe el **if-then** sin **else**.
 - **(if 0/=1 then 3 else 4) + 1** es una expresión válida.
 - Muchos lenguajes tienen expresiones condicionales.
- Los **if** se pueden anidar.

```
signo :: (Ord a, Num a, Num b) => a -> b
signo n = if n < 0 then -1 else
          if n == 0 then 0 else 1
```

Guardas

```
mayor' :: Ord a => a -> a -> a
mayor' x y | x > y      = x
           | otherwise = y

signo' :: (Ord a, Num a, Num b) => a -> b
signo' x | x > 0      = 1
         | x < 0      = -1
         | x == 0     = 0
```

- Las guardas se evalúan de arriba a abajo. El orden importa.
- Se devuelve lo que haya a la derecha de la primera guarda que sea cierta.
- `otherwise` está definido como `True`.
- Se permite solapamiento en las guardas.
- Las guardas no necesitan ser exhaustivas.
 - Si ninguna es cierta, se pasa a la siguiente ecuación.
 - Y si no hay más ecuaciones, error: `Non-exhaustive patterns in function`.

```
estadoAgua :: Double -> String
estadoAgua t | t <= 0      = "hielo"
              | t < 100    = "agua"
              | otherwise = "vapor"

factorial :: Integral a => a -> a
factorial 0 = 1
factorial n | n > 0 = n * factorial (n-1)
```

- `Integral` es mejor aquí que `(Ord a, Num a)`.
- Evita bucles sin fin con argumentos negativos: `factorial (-1)` da error en vez de no terminar nunca.
- Se pueden mezclar patrones, guardas, condicionales...

```
inserta :: Ord a => a -> [a] -> [a]
inserta x [] = [x]
inserta x (y:ys) | x <= y      = x : y : ys
                 | otherwise = y : inserta x ys
```

- Inserta un elemento en una lista ordenada, en su sitio: `inserta 3 [1, 2, 4, 5] ~> [1, 2, 3, 4, 5]`.

case

```
fib :: Int -> Int
fib n = case n of
    0 -> 1
    1 -> 1
    _ -> fib (n-1) + fib (n-2)
```

- Las alternativas son patrones.
- Las alternativas se comprueban de arriba a abajo. El orden importa.
- Las alternativas no necesitan ser exhaustivas.
 - Si ninguna encaja se produce un error: `Non-exhaustive patterns in case`.
- Se permite solapamiento en las alternativas.
- Los casos de un mismo `case` deben empezar en la misma columna.

```
acabaEnA :: String -> Bool
acabaEnA s = case reverse s of
    ('a':_) -> True
    _        -> False
```

```
apariciones :: Int -> String
apariciones n = "Soy " ++ (case n of 1 -> "único"; _ -> "frecuente") ++ "."
```

- El `case`, como el `if`, es una expresión.
- Las alternativas pueden ir en una sola línea, separadas por `;`.

Ejercicios

1. Escribe una función para la conjunción de dos valores booleanos usando `if`.
2. La función `tail` de Haskell toma una lista y devuelve su cola, es decir, lo que queda tras eliminar el primer elemento: `tail [1, 2, 3] ~> [2, 3]`. Pero se produce un error si tratamos de evaluar `tail []`. Define una función `safetail` que se comporta como `tail` sobre listas no vacías y deja las listas vacías tal cual: `safetail [] ~> []`. Define la función de cuatro formas: usando `if`, usando guardas, usando patrones y usando `case`. Si necesitas usar la función predefinida `tail`, hazlo.
3. Partiendo de un entero positivo, si es par se divide entre 2 y si es impar se multiplica por 3 y se le suma 1; con el resultado se repite lo mismo, y así hasta llegar a 1. La *conjetura de Collatz* dice que siempre se llega a 1. Define una función `collatz :: Int -> Int` que devuelva cuántos pasos hacen falta para llegar a 1 desde su argumento. Por ejemplo, `collatz 6 ~> 8`, porque 6, 3, 10, 5, 16, 8, 4, 2, 1. Puedes usar la función `even`.